What Each Verification Method Catches: A Case Study of a Uniswap v4 Token Launchpad

Prince Panwar · Independent technical report, October 2026

Abstract

Smart contracts are usually checked with several kinds of verification at once: unit tests, fuzzing, symbolic execution, static analysis and manual review. Little is known about which of these methods actually finds the defects in a real system, and why the others miss them. This report studies that question on one production system: a token launchpad of about 3,200 lines of Solidity in eight contracts and two libraries at the reviewed version. It sells new tokens along a bonding curve and then moves their liquidity into a Uniswap v4 pool. I designed, implemented and verified the contracts between July and September 2026. Verification used 298 unit tests, 34 stateful fuzz and invariant tests, 6 symbolic proofs (Halmos), two analysers (Slither and Mythril), fork tests against live Uniswap v4 deployments on three chains, and a written internal security review.

From the project's records I traced 22 defects to the method that first exposed each one, and recorded which methods had missed it. The two automated analysers detected none of the defects. The two most severe ones, a liquidity calculation that would have stopped every launch on one chain from graduating and a 13.33% price drop at graduation, were found by fork tests run at each chain's real parameters and by end-to-end price measurement. Manual review found 7 of the 22, including all five medium-severity design and logic defects. I also report three ways verification evidence silently stopped meaning anything: an analyser that only ever examined an inert contract, a test whose expected values came from the same constants as the system it tested, and proofs that timed out but were described as proven. The report closes with practices that prevent these failures.

Keywords: smart contract verification, fuzzing, symbolic execution, static analysis, decentralised finance, Uniswap v4, empirical case study

1 Introduction

A deployed smart contract usually cannot be changed, and it often holds other people's money. Its defects are therefore expensive, and developers verify contracts with several methods at once. A typical project combines unit tests, property-based fuzzing, symbolic execution, static analysis and manual review, and then commissions an external audit.

Each method gives a different kind of evidence. A unit test checks chosen inputs. A fuzzer samples many random inputs. A symbolic tool reasons about all inputs at once, within a time budget. Earlier studies benchmarked analysis tools on large sets of contracts with known or injected bugs [4, 5, 6]. They show that tools miss many real bugs, but they rarely follow one system through its development and ask, defect by defect, what found it and what missed it.

This report does that for one system: a token launchpad that I designed, built and verified, deployed on several EVM chains. It asks three research questions:

- **RQ1.** Which verification method first exposed each defect found in the system?
- **RQ2.** Why did the other methods miss those defects?
- **RQ3.** How can verification evidence itself become wrong, and what practices detect that?

The contributions are:

1. A dataset of 22 defects from one system, each traced to the method that exposed it, with the methods that missed it (Section 5).
2. An analysis of the two most severe defects. Both were invisible to unit tests for structural reasons: one lived in a ratio of parameters that no test sampled, and the other only appeared when two correct computations were compared with each other.
3. A classification of 13 cases in which verification evidence became wrong while still reporting success, and the practices that detect them: canary inputs, vacuity guards, mutation testing and constructor-time cross-checks (Section 6).

This is an independent technical report, not a peer-reviewed publication. It describes my own work on the contracts. The contracts were also audited externally by QuillAudits; that audit is not part of the data analysed here.

2 Background

2.1 Bonding-curve launchpads

A token launchpad lets anyone create a token and sell it without first providing liquidity. The launchpad itself is the only seller: it prices each trade on a bonding curve, a fixed formula that raises the price as more tokens are sold. The curve studied here is a constant-product curve with virtual reserves. With a virtual quote reserve V, real quote collected R and token reserve T, every trade preserves the product k:

$$k = (V + R) \cdot T \qquad (1)$$

When the quote collected reaches a graduation target, the curve closes. The remaining tokens and the collected quote are moved into an ordinary decentralised exchange pool, where trading continues. This step is called graduation. Two properties matter at that moment: the new pool must open at the price where the curve closed, and the transfer must not fail, since a failed graduation strands the launch.

2.2 Uniswap v4 and hooks

Uniswap v4 [8] keeps every pool inside a single PoolManager contract. Liquidity is added through a PositionManager, which computes how much of each asset a position needs from a liquidity value L and the pool's price, and reverts if the caller has not provided enough. A pool can be bound to a hook: a contract that the PoolManager calls before and after swaps. The launchpad uses a hook to charge a high fee immediately after graduation, which then decays, to discourage sniping bots. Uniswap v4 is not deployed at the same addresses on every chain, so each chain has to be confirmed separately.

2.3 Verification methods

The methods compared in this report are:

Method	What it checks	Tool used
Unit tests	Chosen inputs against expected outputs	Hardhat
Stateful fuzzing	Invariants over long random call sequences	Foundry [11]
Symbolic proof	A property for all inputs, within a solver time budget	Halmos [10]
Symbolic execution	Known vulnerability patterns along explored paths	Mythril [3]
Static analysis	Known bug patterns in the source, without running it	Slither [1]
Fork tests	The real contracts against live Uniswap v4 state, copied from a chain	Hardhat network forking
Manual review	Reasoning about design, attacker options and specification	The author

The last two differ in kind. A fork test runs the code against real third-party contracts and real chain parameters instead of mocks. Manual review is not automated at all, and its quality depends on the reviewer.

3 System under study

The system is the launchpad's on-chain core. It is written in Solidity 0.8.24 and compiled with the IR pipeline and optimiser settings that are identical across the test, fuzzing and deployment builds. At the reviewed version it held about 3,200 lines in eight contracts and two libraries; one more contract (LPFeeRatchet) was added later. During the study period it targeted four EVM chains: Base, Ink, Monad and Robinhood Chain. BNB Chain and Polygon were added after the study period.

3.1 Lifecycle

Each launch passes through five phases. A factory creates the token and a bonding-curve clone (UNINITIALIZED to TRADING). Users buy and sell on the curve until the graduation target is reached (CLOSED). The closing purchase triggers graduation in the same transaction: a migration module opens a Uniswap v4 pool at the curve's final price, mints a full-range position, and hands it to a locker that can never withdraw it (GRADUATED). If graduation cannot be settled, the curve enters RECOVERY and sells reopen, so no funds are trapped.

Contract	Role
BondingCurve	The pre-graduation market, one clone per launch
TokenFactory	The only entry point for creating a launch
LaunchToken	The ERC-20 token, fixed supply, minted once
V4MigrationModule	Opens the Uniswap v4 pool and mints the position
LaunchPricing (library)	Computes the opening price and the liquidity it can pay for
LaunchHook	Charges the decaying launch fee on the new pool
LPLocker	Holds the liquidity position permanently and collects its fees
TokenSplitter	Receives every fee and lets creators and the protocol claim them
LPFeeRatchet	Lowers the creator's share of pool fees as the token's value grows
HookDeployer	Places the hook at an address that encodes its permissions

3.2 Trust model

No contract has an owner, a pause function, an upgrade path or a function to sweep funds. There are two privileged roles. A factory admin can change settings for future launches only. A registrar admin can authorise new registrars in the fee splitter. Neither can move funds that have already been credited. The system also trusts three things outside its own code: Uniswap v4 itself, the Permit2 contract, and an optional external contract that can restrict who may buy.

3.3 Properties

The design document states 21 invariants that the tests assert. Three of them matter most for this report:

- **I6:** the curve's product k never decreases, so no round trip of buying and selling can make a profit.
- **I9:** the pool opens at exactly the price where the curve closed (graduation continuity).
- **I16:** a closing purchase either completes graduation or reverts; it never leaves the launch half-settled.

4 Methodology

This is a retrospective single-case study [13]. The data is the project's own record: the Git history (117 commits, July to October 2026), the internal security review, the design document and the comments inside the test suites. No defect was added or reconstructed for this report. The dataset covers development up to the close of the internal review on 18 August 2026; later observations are reported separately.

4.1 Verification setup

The final state of verification, measured on 18 August 2026, was:

Method	Scale	Result
Unit tests	298 tests, run at four chains' parameter scales	All pass
Stateful fuzzing	34 tests in 6 suites, 32,768 calls per invariant	All pass
Symbolic proof (Halmos 0.3.3)	6 properties	4 proved, 2 timed out

Method	Scale	Result
Static analysis (Slither 0.11.6)	102 detectors over 103 contracts	41 results, 0 High
Symbolic execution (Mythril 0.24.8)	9 contracts	No issues
Fork tests	25 to 26 tests per chain, live Uniswap v4 on Base, Ink and Monad	All pass
Manual review	Per-contract checklist: access control, arithmetic and rounding, reentrancy, external calls, ETH handling, initialisation	20 findings in the review

Two setup choices matter for the results. First, since 1 August 2026 the whole unit suite can run at each chain's real graduation target, which differs by a factor of about 90,000 in native units between Base (4.2 ETH) and Monad (380,000 MON). As Section 5.2 shows, this alone was not enough. Second, the analysers were checked before their results were accepted: Mythril was run on a deliberately broken "canary" contract, compiled with the same settings, to confirm it could detect a known reentrancy bug.

4.2 Defect selection

A defect is included if the record shows that the system or its verification evidence was wrong at some point, and that this was discovered and recorded. Two groups are distinguished:

- **Group A, defects in the system (9):** behaviour that was wrong or unsafe, including configuration and design weaknesses. This covers the 2 most severe defects found during development and 7 from the security review.
- **Group B, defects in the evidence (13):** cases where a test, proof, tool result or reported figure no longer meant what it claimed, while the system itself might be correct.

Findings recorded as accepted by design (for example, that assets sent to the splitter by mistake cannot be recovered), purely commercial items, and around 40 documentation errors were excluded. The documentation errors are discussed briefly in Section 6.

4.3 Classification

For each defect I recorded the method that first exposed it, the methods that were in place at the time and did not expose it, and the reason they did not. The detecting method was taken from commit messages and the review text, not inferred. Severity uses the

review's scale (impact × likelihood, where likelihood includes operator error). The two development-time defects were not rated in the review; I rate them Critical and High.

5 Results

None of the 22 defects was first exposed by the static analyser, by Mythril, by the stateful fuzzer or by the unit tests. Manual review exposed 7, re-running and re-measuring earlier checks exposed 4, and the remaining 11 came from eight other methods, each finding one or two (Table 1 and Figure 1). The fuzzer and the unit tests did their work after discovery instead: every Group A defect that was fixed in code now has a regression test or a fuzz property.

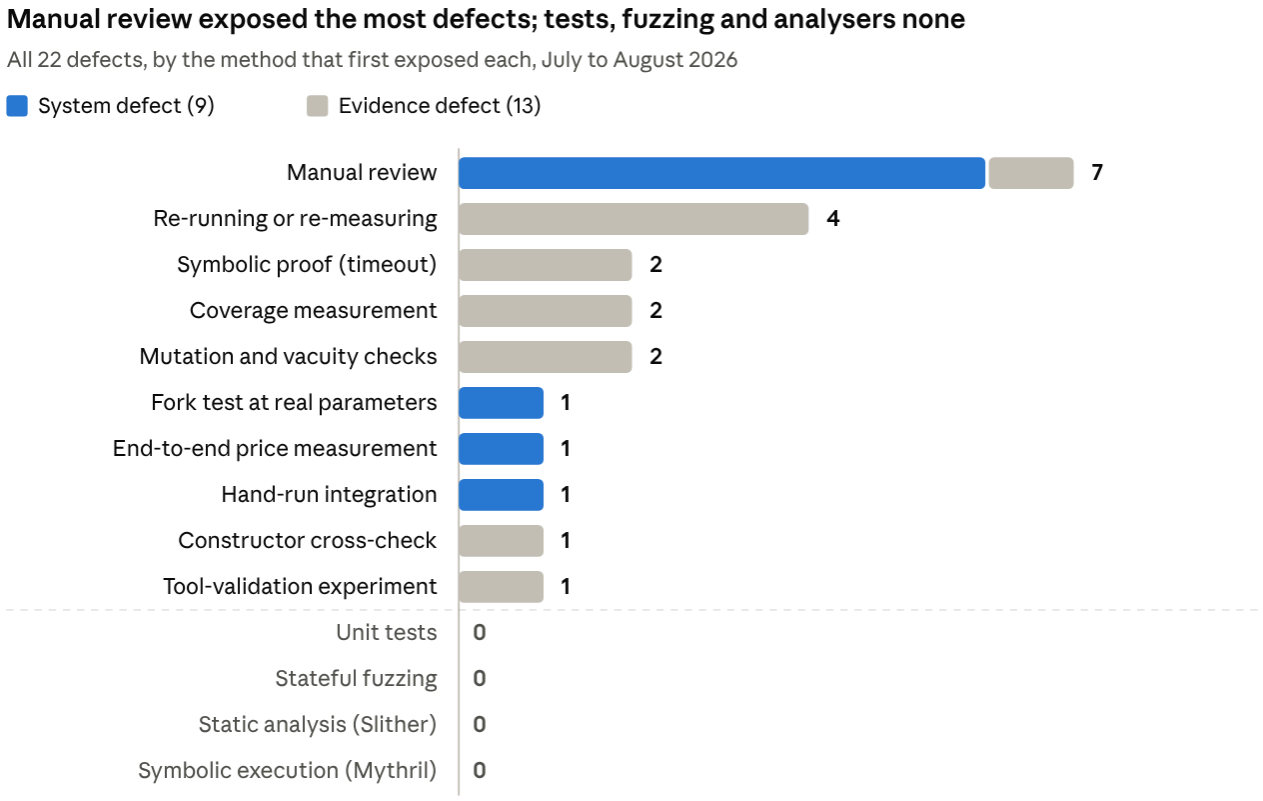


Figure 1 · defects by the method that first exposed them · from Table 1

Blue bars are defects in the system's behaviour (Group A); grey bars are defects in the verification evidence (Group B). The methods below the dashed line ran throughout the study period and exposed no defect first.

5.1 The defects

Table 1. The 22 defects, the method that first exposed each, and why earlier methods missed it. IDs beginning M, L or I are from the internal review; D1 and D2 were found during development.

ID	Defect	Group	Severity	Exposed by	Why earlier methods missed it
D1	Liquidity formula fails at Monad's scale; every Monad launch would end in recovery	A	Critical	Fork test at the chain's real target	Mock ignored the liquidity value; unit tests used hand-picked amounts, and the defect lived in a ratio
D2	Pool opened 13.33% below the curve's closing price	A	High	End-to-end price measurement	Every individual trade was correct; no test compared the two prices
M-1	Compromised registrar admin can run attacker code inside the locker	A	Medium	Manual review from the attacker's viewpoint	Listing the locker's outgoing calls misses calls made back into it
M-2	Anti-sniping buy caps shipped disabled; one 4.2 ETH buy graduated a token	A	Medium	Manual review of deployment	Test fixtures deliberately disabled the caps
M-3	Unbounded virtual reserve lets a factory deploy cleanly, then revert every trade	A	Medium	Manual review, confirmed by a constructed deployment	Shipped parameters are far below the overflow point
M-4	Fee ratchet read a spot price at a moment the attacker chose	A	Medium	Manual review from the attacker's viewpoint	Tests modelled an attacker holding a price, not choosing when it is read
M-5	Ratchet's confirm rule could be stalled indefinitely	A	Medium	Manual review, confirmed by simulation	Added as a defence against price manipulation and reviewed only as such
L-11	Two contracts held one constant with nothing checking they agree	A	Low	Manual review	No test referenced both values
I-8	First integrator's test harness disabled the buy caps	A	Info	Hand-run integration	Outside the repository; the contracts behaved as configured
L-1	Three of four width guards had no test	B	Low	Coverage measurement	Coverage had never been run
L-1b	A new guard added by the M-3 fix had no test	B	Low	Coverage measurement	Introduced by the fix itself
L-2	Mythril results predated the current code	B	Low	Re-running the tool	Results were cited, not re-run

ID	Defect	Group	Severity	Exposed by	Why earlier methods missed it
L-3	Slither result counts had drifted (29 claimed, 33 measured)	B	Low	Re-running the tool	Figures were copied between documents
L-7	Liquidity bound described as proven; the proof times out	B	Low	Symbolic proof (timeout)	A timeout had been read as a pass
L-8	Monad fork evidence not reproducible; pinned block was pruned	B	Low	Re-running the fork suite	The public node keeps about 10 days of state
L-9	Mythril only ever analysed an inert contract	B	Low	Tool-validation experiment	A clean report looks the same as a vacuous one
L-10	Fork suite validated fee constants against themselves	B	Low	Constructor cross-check (L-11 fix)	Expected values came from the same stale constants
L-12	Floored-average property times out	B	Low	Symbolic proof (timeout)	Cost of 256-bit signed division
I-3	Test counts, coverage figures and open items stale	B	Info	Re-measurement	Reported, not measured
I-5	2 of 27 "fuzz tests" assert nothing	B	Info	Manual inspection	Counted by the test runner
V-1	A fuzz suite passed without exercising its target (first of two)	B	Low	Mutation and vacuity checks	Invariants hold trivially if nothing happens
V-2	A fuzz suite passed without exercising its target (second of two)	B	Low	Mutation and vacuity checks	Invariants hold trivially in an empty state

V-1 and V-2 come from one sentence in the review: two fuzz suites were caught being vacuous during development, by the mutation and vacuity checks every suite must pass. The record does not say which suites, so they are counted together under that method.

5.2 Case study: a defect that lived in a ratio (D1)

When a launch graduates, the migration module must tell the PositionManager how much liquidity L to mint from the quote amount x and token amount y it holds. The original code used the geometric mean:

$$L = \sqrt{(x \cdot y)} \qquad (2)$$

This is exact only if the pool's square-root price is exact. The code computed that price in fixed-point arithmetic and rounded it down, so L came out slightly larger than the amounts could pay for. The shortfall grows as the quote asset gets cheaper relative to the token. At Base's target of 4.2 ETH against about 195 million tokens it was about 2 wei and was absorbed by the position's price range. At Monad's target of 380,000 MON it was about 4.5×10^7 wei, and the PositionManager reverted. Because the closing purchase defers graduation when settlement fails, every launch on Monad would have ended in recovery and never reached a pool.

The unit tests missed this for two independent reasons. The mock PositionManager ignored the liquidity argument and simply charged the maximum amounts, so the liquidity calculation had no effective coverage at any scale. And every test used one of a few hand-picked amount pairs; the defect depended on the ratio between the amounts, which those pairs never reached.

The defect was exposed on 2 August 2026, one day after Monad's target was set, by the fork suite running against live Uniswap v4 at that target. The fix derives L from the price actually being set, which is exact at any scale. Three changes then made the class visible to cheaper methods: the mock now prices a mint with Uniswap v4's own math libraries; a fuzz test checks 10,000 random amount pairs; and its property is stated so that it survives future changes:

> Migration either succeeds or fails with one of the module's own named errors, never with the PositionManager's insufficient-amount error.

A symbolic proof of the same bound was written as well, and it does not complete (L-7). The bound is therefore strongly evidenced, but not proven.

5.3 Case study: two correct computations that disagreed (D2)

The curve's closing price is set by its reserves. The pool's opening price was set separately, by the amounts handed to it at graduation. Both were computed correctly, but nothing required them to agree. With a graduation fee f, curve supply C, pool supply Q, virtual reserve V and graduation target G, the ratio of the two prices was

$$P(\text{pool}) \,/\, P(\text{curve}) = (1 - f) \cdot (C \,/\, Q) \cdot V \,/\, (V + G) = 0.975 \cdot 4 \cdot 1.2 \,/\, 5.4 = 0.8667 \qquad (3)$$

Every token therefore lost 13.33% of its price at the moment it graduated: 2.5% from the fee and 11.11% from the structure itself. No unit test failed, because no single trade was wrong. The defect was exposed on 29 July 2026 by taking a launch through graduation and comparing the two prices: the curve closed at 2.3625×10^{-8} and the pool opened at 2.0475×10^{-8}.

The fix turns the hidden assumption into a constraint. The fee is now charged on both sides in equal proportion, so it cancels out of the ratio, and setting the remaining ratio to 1 gives

$$Q = C \cdot V / (V + G) \qquad (4)$$

so the supply split is derived from the curve instead of being chosen, and the factory's constructor rejects any configuration that breaks it. The design had seemed to have two free parameters; it has one. The property is now asserted directly as invariant I9, at several fee levels and along three paths to graduation. The constraint later prevented a regression: a planned change to a flat graduation fee would have reopened a gap of 1.19%, and it was implemented by burning the same fraction of tokens so the ratio stays exact.

5.4 Case study: a mitigation with its own defect (M-4, M-5)

A later contract, the fee ratchet, lowers the creator's share of pool fees once a token's value crosses fixed tiers. The change is permanent, so the price it observes must be hard to manipulate. The first design read the spot price, waited 24 hours, read it again, and committed only if both readings agreed. The design document claimed that an attacker would have to hold a false price for a full day.

Manual review showed the claim was wrong (M-4). The function that takes the reading is public, so the attacker chooses when the price is read, not only what it is. Buying, triggering the reading and selling can all happen in one transaction, twice, a day apart. The estimated cost was about $1,900 to cut a creator's share from 70% to 40% permanently.

The agreement rule, which was the design's defence against manipulation, had its own defect (M-5). A creator could restart the 24-hour clock every day with one small trade, and a token whose price merely moved across a tier boundary could fail to commit with no attacker at all. A simulation held a token in the top tier for almost all of six days across twelve readings, and the split never moved. Both were fixed by replacing the two readings with a time-weighted average of the price, accumulated by the pool's hook on every swap. The measured cost is 2,799 gas per swap.

The lesson of M-5 is about review scope. The agreement rule was reviewed only as a defence against price manipulation, and its failure is the mirror image of that attack: instead of forcing a change, it prevents one. A reviewer checking a mitigation tends to ask whether it stops the attack, not what new behaviour it adds.

5.5 What the automated analysers contributed

Within the study period Slither reported 41 results and Mythril none, and neither exposed a defect in the dataset. Their contribution was negative evidence: they make some classes of bug, such as known reentrancy patterns, unlikely. Even that evidence needed

checking. Mythril's report on the bonding curve was vacuous (L-9). The deployed implementation contract deliberately starts in a finished state, so that nobody can use it directly, and every function reverts immediately. Mythril finished in 18 seconds of an 1,800-second budget. A copy with that one line removed ran for 219 seconds, twelve times longer, and was also clean. Only that second run says anything about the curve's live logic.

Two later observations, from September 2026 and outside the dataset, point the same way. When the hook was rewritten, Slither flagged three real issues in it, all minor: two state updates or events placed after external calls, and a failure that was silently ignored. All three were fixed. In the same rewrite, Mythril's earlier clean result for the hook turned out to be vacuous for a different reason: the hook's constructor checks that its own address encodes its permissions, Mythril deployed it at an address that did not, and the constructor reverted after 7 seconds with nothing left to analyse.

6 Discussion

6.1 RQ1 and RQ2: why the cheap methods missed the serious defects

The two most severe defects share a cause. Each needed a combination of values that the cheap methods did not produce. For D1 it was a ratio between two amounts, reached only at one chain's real parameters. For D2 it was a comparison between two values that were each correct on their own. Unit tests check the values someone thought to choose. Fuzzers sample inputs, but within bounds the test author sets, and they check only the properties the author wrote down. Neither can find a property that nobody has stated yet.

The methods that did find them were, in a sense, less clever. A fork test ran the real code against real third-party contracts with production parameters. An end-to-end measurement took one launch through its whole life and compared two numbers. What they share is realism: real dependencies instead of mocks, real parameters instead of convenient ones, and whole flows instead of single functions.

Manual review found every medium-severity design defect. Its advantage was the point of view. M-1 and M-4 were found by reasoning forward from what an attacker controls, not backward from what the code does. In M-1, a list of the locker's safety conditions was found incomplete four times, and each omission was found that way. The weakness of manual review is its author: the rule behind M-5 was designed and reviewed by the same person, who checked it only against the attack it was written for.

6.2 RQ3: how evidence goes wrong

Thirteen of the 22 defects were in the evidence, not the system. They fall into four patterns:

Pattern	Defects	What happened	Practice that detects it
Vacuous success	L-9, V-1, V-2, I-5	A check passed because it never exercised its target	Canary contracts with planted bugs; vacuity guards that count how often the target was reached; comparing run time to budget
Self-validation	L-10	Expected values came from the same constants that configured the system	Import expectations from a single shared source; cross-check related constants in the constructor
Overstated proof	L-7, L-12	A solver timeout was read or documented as a proof	Report timeouts separately; keep the fuzz test that covers the same property
Drift and decay	L-1, L-1b, L-2, L-3, L-8, I-3	Figures and results were copied forward instead of re-measured, or the environment changed	Re-run before citing; pin fork blocks and record the expiry; measure coverage after each fix

Vacuous success is the most dangerous pattern, because it looks identical to real success. The only reliable defence found here was to test the test: plant a defect the tool must catch (the canary), or count how often the property was actually exercised. A canary is necessary but not sufficient. Mythril passed its canary test, yet its bonding-curve result was still vacuous (L-9); comparing its run time with its budget is what exposed that.

The documentation showed the same pattern at a larger scale. Around 40 documentation errors were found during review, two of them security claims stated backwards. No tool reads comments, so none of these could be found automatically.

6.3 Practical recommendations

1. Run the whole test suite at every production parameter set, not only the convenient one. The graduation targets behind D1 differ by a factor of about 90,000 in native units between two chains.
2. Write mocks that enforce the same arithmetic as the real dependency, or test against the real dependency on a fork.
3. State cross-component properties explicitly, such as "the pool opens at the curve's closing price", and test them directly.

4. Test every check: plant a canary for analysers and count exercised states in fuzz suites.
5. When reviewing a mitigation, review it as new code, including what it prevents.
6. Re-measure every figure before citing it.

6.4 Threats to validity

- **Single system.** The results come from one codebase of about 3,200 lines. They show that these failure modes occur, not how often they occur elsewhere.
- **Author bias.** I wrote the code, the tests and the review. A reviewer who shares the author's mental model shares their blind spots. The external audit by QuillAudits is an independent check on the system, but its results are not part of this analysis.
- **Attribution.** Detection methods were taken from commit messages and the review text. Where a defect was found by a person running a tool, I credited the tool's output, not the person's decision to run it.
- **Small counts.** With 22 defects, the counts per method are not statistically meaningful. The value of the study is in the mechanisms of each case, not in the totals.
- **Survivorship.** Only defects that were found are counted. Defects that every method missed are, by definition, absent.

7 Related work

Analysis tools. Slither [1] is a static analysis framework that matches known bug patterns on an intermediate representation of Solidity. Mythril [3] and Oyente [2] use symbolic execution to search for vulnerable execution paths. Echidna [12] introduced property-based fuzzing for smart contracts, and Foundry [11] and Halmos [10] brought stateful fuzzing and symbolic testing into the developer's own test framework.

Evaluations of tools. Durieux et al. [4] ran nine analysis tools on 47,587 Ethereum contracts and found low agreement between them and many missed vulnerabilities. Ghaleb and Pattabiraman [5] injected bugs into contracts and found that static analysers missed many of them. Zhang et al. [6] studied exploitable bugs from real audits and found that most of them were not detectable by existing tools, because they depended on the application's own logic. Chaliasos et al. [7] compared tools with real DeFi attacks and found that tools would have prevented only a small share of the losses. The present report agrees with these results, but approaches the question from the other side: instead of many contracts and one tool run each, it follows one system through its development and records, for each defect, which method actually exposed it.

DeFi attacks. M-4 belongs to the class of price-manipulation attacks that rely on reading a price at a moment the attacker controls. Qin et al. [9] showed how flash loans make such manipulation cheap within a single transaction. The time-weighted average used for the

fix follows the oracle design of Uniswap v3 [15].

Test quality. Mutation testing [14] measures whether tests detect deliberately introduced faults. The vacuity and canary checks in Section 6.2 apply the same idea to fuzz suites and to analysis tools.

8 Conclusion

In this system, the methods that are cheapest to run (unit tests, fuzzing and the two analysers) found none of the 22 defects first. The serious defects were found by realism: running real code against real dependencies at real parameters, and measuring a whole flow end to end. Manual review from the attacker's point of view found every medium-severity design defect, though the same author's design also produced one of them (M-5). More than half of the defects were in the verification evidence itself, and the most dangerous of them looked exactly like success.

The practical conclusion is not that automated tools are useless. They ran continuously, they rule out known bug classes, and the tests now guard every fixed defect against regression. It is that their output is evidence only after it has been checked: a test should be shown to fail on a planted defect, a solver timeout should be reported as a timeout, and a figure should be measured before it is cited. Future work could apply the same defect-by-defect tracing to several projects, to see which of these patterns generalise.

Acknowledgements

Arpit contributed to the repository. The contracts were audited externally by QuillAudits. An AI writing assistant was used to help structure and edit the text; the system, the verification work, the data and the conclusions are my own.

Data availability

The contract source code and the internal security review are available from the author on request. Findings that remained open at the end of the study period are summarised here without exploitable detail.

References

1. J. Feist, G. Grieco, A. Groce. Slither: A Static Analysis Framework for Smart Contracts. In *Proc. 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, IEEE, 2019.

2. L. Luu, D.-H. Chu, H. Olickel, P. Saxena, A. Hobor. Making Smart Contracts Smarter. In *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2016.
3. B. Mueller. Smashing Ethereum Smart Contracts for Fun and Real Profit. *HITB Security Conference*, Amsterdam, 2018. Tool: Mythril, https://github.com/ConsenSys/mythril
4. T. Durieux, J. F. Ferreira, R. Abreu, P. Cruz. Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts. In *Proc. 42nd International Conference on Software Engineering (ICSE)*, 2020.
5. A. Ghaleb, K. Pattabiraman. How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection. In *Proc. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2020.
6. Z. Zhang, B. Zhang, W. Xu, Z. Lin. Demystifying Exploitable Bugs in Smart Contracts. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023.
7. S. Chaliasos, M. A. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, B. Livshits. Smart Contract and DeFi Security Tools: Do They Meet the Needs of Practitioners? In *Proc. 46th International Conference on Software Engineering (ICSE)*, 2024.
8. H. Adams et al. Uniswap v4 Core. Whitepaper, Uniswap Labs, 2024.
9. K. Qin, L. Zhou, B. Livshits, A. Gervais. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security (FC)*, 2021.
10. a16z. Halmos: symbolic testing for EVM smart contracts. https://github.com/a16z/halmos
11. Foundry: a toolkit for Ethereum application development. https://github.com/foundry-rs/foundry
12. G. Grieco, W. Song, A. Cygan, J. Feist, A. Groce. Echidna: Effective, Usable, and Fast Fuzzing for Smart Contracts. In *Proc. ISSTA*, tool demonstration, 2020.
13. R. K. Yin. *Case Study Research and Applications: Design and Methods*, 6th ed. SAGE, 2018.
14. M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, M. Harman. Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, vol. 112, 2019.
15. H. Adams, N. Zinsmeister, M. Salem, R. Keefer, D. Robinson. Uniswap v3 Core. Whitepaper, Uniswap Labs, 2021.